

AutoGen: bütün yüzey

İki kılavuz, iki katman, tek belge. Önce AgentChat — hazır ajanlar, beş takım tipi, on bir sonlandırma koşulu. Sonra core — aktör modeli, topic, abonelik, müdahale kapısı, dokuz desen. Her sayfa, her sınıf, her tuzak — çizimle. Ve ölçülen yerler: neyin kaç token'a mal olduđu, hangi desenin sessizce veri kaybettiđi.

kaynak • docs/08-...agentchat-user-guide.md (49.666 kelime) + docs/05-...core-user-guide.md
microsoft/autogen, MIT

özet • docs/10-agentchat-turkce.md + docs/11-core-guide-turkce.md • atıflar 08:NNNN /
05:NNNN

sürüm • autogen-agentchat 0.7.5 (.venv'de doğrulandı) • bakım modunda

ölçüm • poc/kiyas.py • pipeline/compare_fanin.py • pipeline/probe_llm.py

0 - Bu belgenin kuralı

Kılavuz anlatımı ile ölçüm aynı tonda durmamalı. Her iddia nereden geldiğini taşıyor.

ETİKET	ANLAMI
ÖLÇÜLDÜ	Bu projede kod koşturularak elde edildi; ölçüm dosyası gösterilebilir
KAYNAK	Kılavuzun kendi metninden ya da kurulu paketten doğrulandı
TEYİTSİZ	Okunandan; koşturulmadı

Önce bu: AutoGen **bakım modunda**. Kendi README'sinden **KAYNAK** : "AutoGen is now in maintenance mode. It will not receive new features or enhancements... New users should start with Microsoft Agent Framework." Bu kılavuz doğru ama **donmuş**. Buradaki her şey çalışıyor; hiçbirisi gelişmeyecek.

Bir filtre – okumaya başlamadan

Bir eğitim materyalinde `ConversableAgent` ya da `initiate_chat` görüyorsan, o kaynak **v0.2 veya AG2 Classic** anlatıyor ve bu kılavuzla uyumsuz. Hazır kurs değerlendirirken ilk bakılacak şey budur.

İçindekiler

KISIM I – AgentChat

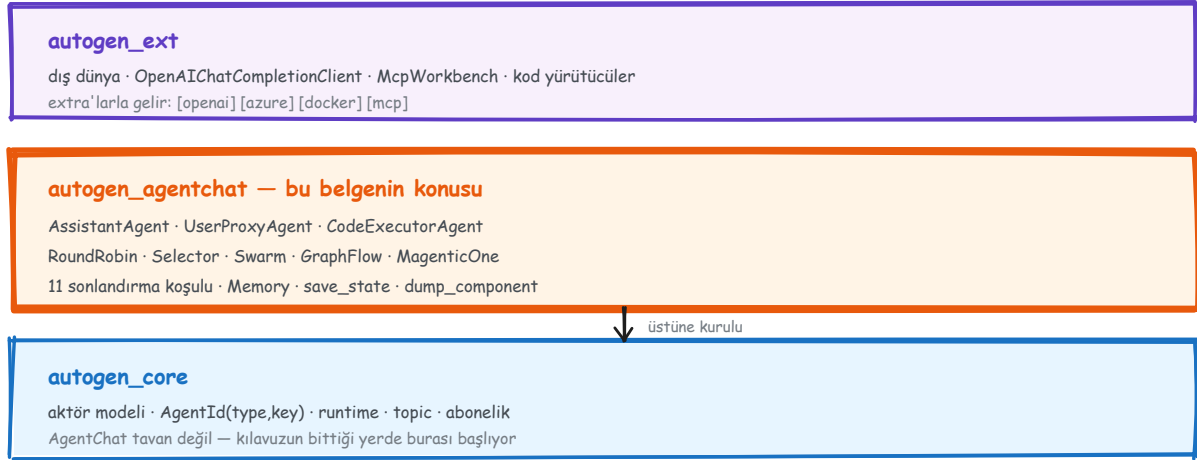
- 1 AgentChat nerede durur
- 2 v0.2 → v0.4: yeniden yazım
- 3 Öğreticinin merdiveni
- 4 Model istemcileri
- 5 Mesajlar: iki aile
- 6 AssistantAgent anatomisi
- 7 Beş takım tipi
- 8 Sonlandırma: on bir koşul
- 9 İnsan döngüde: iki yol
- 10 Durum yönetimi
- 11 GraphFlow derinlemesine
- 12 Bellek ile bağlam farkı
- 13 Günlükleme: iki logger
- 14 Bileşen serileştirme
- 15 Özel ajanlar
- 16 Üç uçtan uca örnek

KISIM II – core

- 17 Neden aşağı inersin
- 18 Ajan kimliği ve yaşam döngüsü
- 19 Topic ve abonelik
- 20 Üç mesajlaşma biçimi
- 21 RoutedAgent: tip önce, sonra koşul
- 22 Runtime ve kendininkini kurmanın bedeli
- 23 Bileşenler: bağlam, tool, workbench
- 24 Müdahale kapısı
- 25 Dokuz desen ve toplama meselesi
- 26 Kullanmadığımız üç yüzey
- 27 Cookbook: on dört tarif
- 28 Kılavuzun kapattığı sorular
- 29 Ölçümler, tek sayfada
- 30 Bu projede ne kullanılıyor

1 • AgentChat nerede durur

08:53 Core'da her şeyi kendin bağlarsın: aktörler, mesaj tipleri, abonelikler. AgentChat'te hazır ajanlar ve hazır takımlar vardır — "iki ajan sırayla konuşsun" üç satırdır.



Üç katman. AgentChat ortada durur ve aşağıya inmen *gerekebilir* — bu projede paralel dal kaybını çözmek için indik (§11).

Kılavuzun sunduğu şeyler

Ajanlar — AssistantAgent, UserProxyAgent, CodeExecutorAgent

Takımlar — RoundRobinGroupChat, SelectorGroupChat, Swarm, GraphFlow, MagenticOneGroupChat

Ayrıca — sonlandırma koşulları, bellek, durum kaydı, bileşen serileştirme

Kurulum • 08:222

```
pip install -U "autogen-agentchat" "autogen-ext[openai]"
```

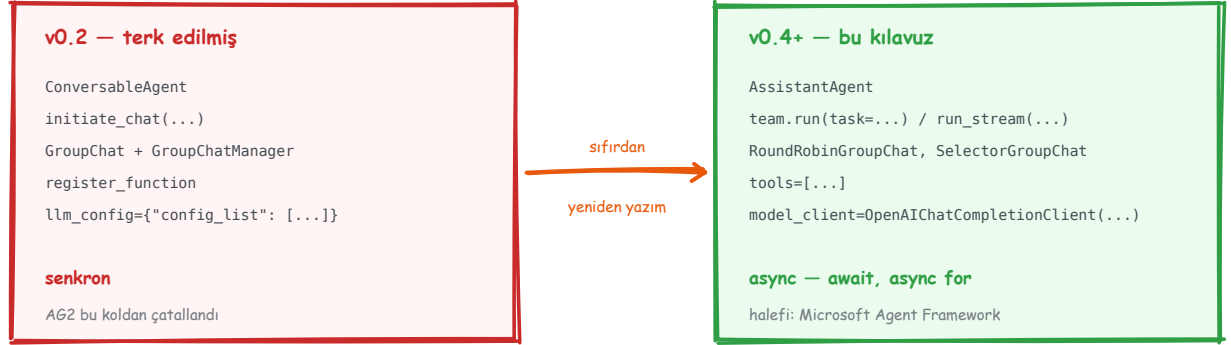
ÖLÇÜLDÜ Bağımlılık tuzağı. autogen-ext'in mcp bağımlılığı **üst sınırsız** yazılmış (mcp>=1.11.0). MCP SDK 2.0 çıkınca ImportError: RequestContext alıyorsun ve düzeltecek kimse yok — bakım modundaki proje üst sınır koymaz. requirements.txt'e mcp>=1.24,<2 pini şart.

2 · v0.2 → v0.4: devam değil, yeniden yazım

08:403 Kılavuzun **en uzun sayfası** (1382 satır) ve bu projenin tezi için en değerlisi.

KAYNAK "...from-the-ground-up rewrite adopting an **asynchronous, event-driven architecture** to address issues such as **observability, flexibility, interactive control, and scale.**"

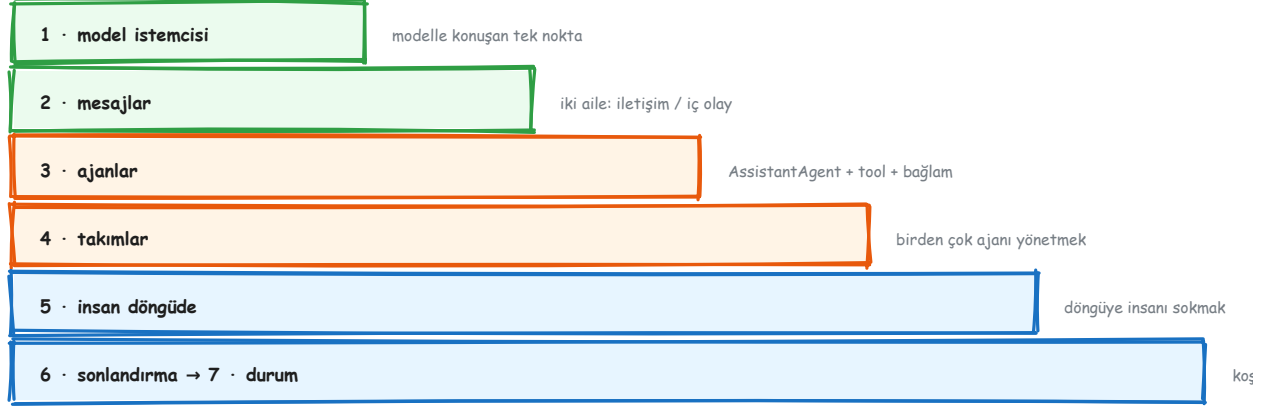
Sayılan dört ihtiyaç — gözlemlenebilirlik, esneklik, etkileşimli kontrol, ölçek — dördü de **üretimin** istediği şeyler. Araştırma kodunun ürün olmaya çalışırken çarptığı duvarın listesi bu.



Aynı isim, ayrı program. Eşleme tablosu bir göç kılavuzu değil, bir *uyumsuzluk* ilanı.

3 · Öğreticinin merdiveni

08:1786 Sıra keyfi değil: her basamak öncekinin üstüne biniyor. Bir basamağı atlarsan üsttekiler havada kalır.



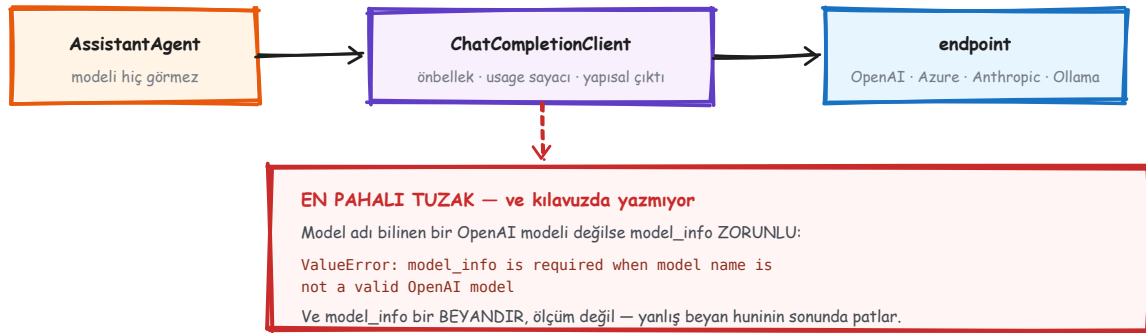
Yedi basamak. Kılavuzun sırası budur ve bu belge de onu izliyor.

4 · Model istemcileri

08:1873 Ajan modelle doğrudan konuşmaz; arada bir `ChatCompletionClient` vardır.

```
from autogen_ext.models.openai import OpenAIChatCompletionClient
model_client = OpenAIChatCompletionClient(model="gpt-4o")
```

Kapsanan istemciler: OpenAI, Azure OpenAI (`AzureTokenProvider` ile AAD kimlik doğrulama dahil), Azure AI, Anthropic, Ollama, ve **OpenAI-uyumlu** her endpoint (`base_url` vererek). Sayfanın anlattığı diğer şeyler: **önbellek** (`ChatCompletionCache` — aynı istemi iki kez ödememek), **kullanım sayacı** (`total_usage()`) ve **yapılandırılmış çıktı**.



"OpenAI-uyumlu endpoint" kullanan **herkes** bu tuzağa düşer. Bizde ilk canlı istekte patladı. → 06 §3 · `pipeline/probe_llm.py`

5 • Mesajlar: iki aile

08:2236 Ayırım pratikte şuna yarıyor: akışı dinlerken "bu bir cevap mı, yoksa ajanın yaptığı bir iş mi" sorusunu **tip** söylüyor.

BaseChatMessage

ajanlar arası İLETİŞİM

TextMessage
MultiModalMessage
HandoffMessage
StopMessage
ToolCallSummaryMessage
StructuredMessage[T]

BaseAgentEvent

ajanın İÇ OLAYLARI

ToolCallRequestEvent
ToolCallExecutionEvent
ModelClientStreamingChunkEvent
MemoryQueryEvent
UserInputRequestedEvent

arayüzde tool çağrılarının ayrı satır olması bu sayede

TUZAK • Takım, tanımadığı mesaj tipini YÖNLENDİRMEZ

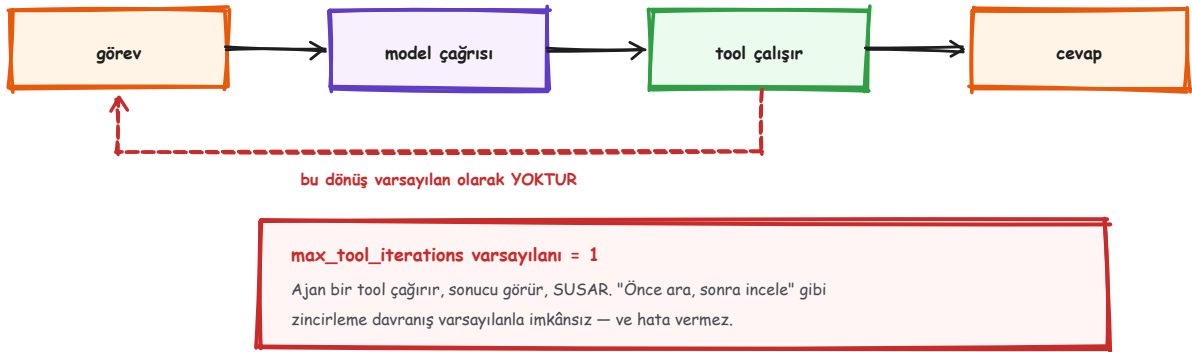
GraphFlow(..., custom_message_types=[StructuredMessage[Score]])

İki aile, iki iş. Yapısal çıktı üreten bir ajan takımın içindeyse tipi *beyan etmen* gerekir — tek başına `agent.run()`'da gerekmez, sorun yalnız yönlendirmede çıkar. → 06 §9

6 • AssistantAgent anatomisi

08:2298 Kılavuzun ana kahramanı. Parametreleri, davranışın tamamını belirliyor.

PARAMETRE	NE YAPAR
model_client	Hangi model — ajan başına ayrı olabilir; kademelendirme buradan
tools	Fonksiyonlar; şema imzadan ve docstring'den üretilir
workbench	Tool <i>kaynağı</i> (MCP dahil) — <code>tools</code> ile birlikte verilemez
system_message	Rolü
description	Takım içinde yönlendirme bunu okur
model_context	Bellek — yoksa ajan turlar arası durumsuzdur
output_content_type	Pydantic ile yapısal çıktı
reflect_on_tool_use	Tool sonucunu yorumlasın mı; <code>False</code> ise ham çıktı cevap olur
max_tool_iterations	Varsayılan 1 — zincirleme tool çağrısı yok
handoffs	Devir hedefleri (Swarm için)
model_client_stream	Token token akış; olmadan cevap tek parça gelir



Tool döngüsü varsayılan olarak **bir tur**. Sessizce erken susan bir ajan, bozuk bir ajandan daha zor fark edilir. → 06 §10

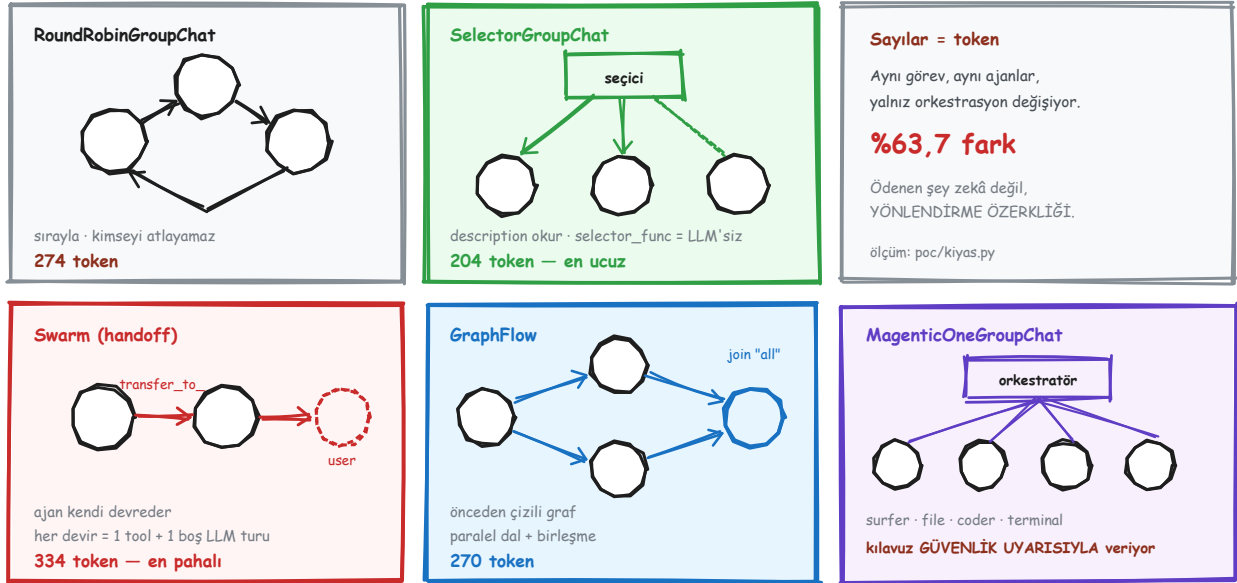
İkinci tuzak: `tools=` ile `workbench=` aynı ajana verilemez — `ValueError: Tools cannot be used with a workbench`. İkisini birlikte istiyorsan yerel fonksiyonları `StaticWorkbench`'e sarıp **liste** ver. → 06 §1

Çalıştırma iki biçimde

```
result = await agent.run(task="...")          # tek seferde
async for event in agent.run_stream(task="..."): # olay olay
    ...
```


7 • Beş takım tipi

08:2813 • 4231 • 4991 • 5398 • 6031 Aralarındaki fark **yönlendirmeyi kimin yaptığı**. Ve ödenen bedel de tam olarak o.



ÖLÇÜLDÜ Beş topoloji ve faturası. "Hangi framework daha iyi" sorusu aslında "hangi yönlendirme maliyetini ödemeye razısın" sorusudur.

Selector'ün iki kaçış kapısı

`selector_func` — seçimi **Python'la** yaparsın, model hiç çağrılmaz. En ucuz yol; bizim triyajımızın mantığı bu. • `candidate_func` — modele sunulacak aday listesini daraltırsın.

Tuzak: `description` boş bırakılmış bir ajan seçiciyi **kör** bırakır. Seçici elindeki tek bilgiyle karar veriyor. → 06 §13

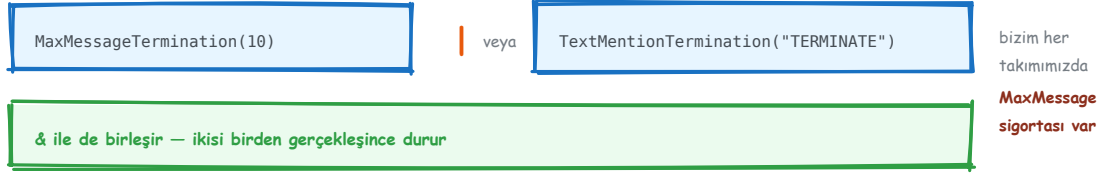
Swarm'ın adlandırma tuzağı

Devir tool'unun adı **küçük harfe düşer** (`transfer_to_flights_refunder`). Elle yazarsan eşleşmez; `Handoff(target=X).name` ile üret. `HandoffTermination(target="user")` ile insana devredince takım durur.

8 • Sonlandırma: on bir koşul

08:3670 **Opsiyonel değil.** Sonlandırma koşulu olmayan bir takım, sonsuz döngüye girdiğinde gerçek bir fatura üretir.

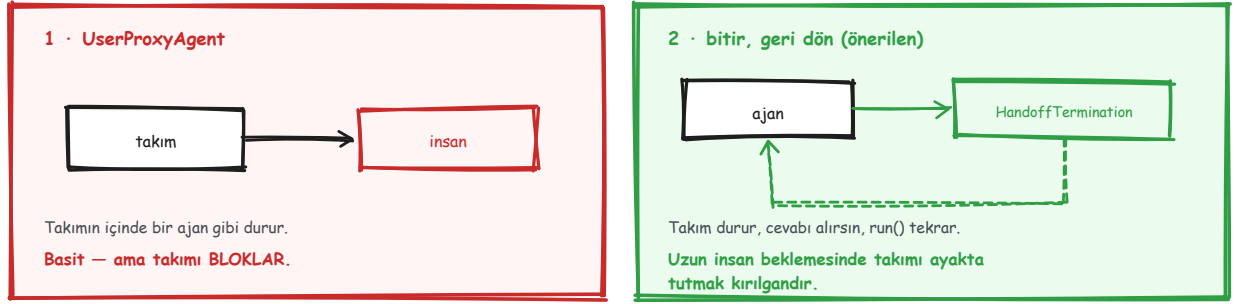
KOŞUL	NE ZAMAN DURUR
<code>MaxMessageTermination</code>	N mesaj sonra
<code>TextMentionTermination</code>	Bir metin geçince ("TERMINATE")
<code>TokenUsageTermination</code>	Token sınırı
<code>TimeoutTermination</code>	Süre
<code>HandoffTermination</code>	Belirli bir hedefe devir olunca
<code>SourceMatchTermination</code>	Belirli bir ajan konuşunca
<code>ExternalTermination</code>	Dışarıdan <code>set()</code> çağrılınca
<code>StopMessageTermination</code>	<code>StopMessage</code> gelince
<code>FunctionCallTermination</code>	Belirli bir tool çağrılınca
<code>FunctionalTermination</code>	Kendi yazdığın fonksiyon



Koşullar `|` ve `&` ile birleştirilebilir. Dönen `TaskResult` iki şey taşır: `messages` ve `stop_reason`.

9 · İnsan döngüde: iki yol

08:3327 Kılavuz ikincisini öneriyor, ve sebebi net.



İkisi de çalışır; farkı, insan ne kadar beklerse o kadar büyür.

10 • Durum yönetimi

08:4007

```
state = await agent.save_state()      # veya team.save_state()
await agent.load_state(state)
```

Kaydedilen şey ajanın **bağlamıdır** — yani `model_context` 'in tuttuğu. Bağlam vermediysen kaydedilecek sohbet de yoktur. Bu yüzden `model_context` ve `save_state` birlikte düşünülmeli.

Bizde: Sohbet arayüzünde sunucu yeniden başlayınca konuşmanın kalması bu mekanizma.
`pipeline/conversation.py` , `gateway/context_engine.py`

11 · GraphFlow derinlemesine

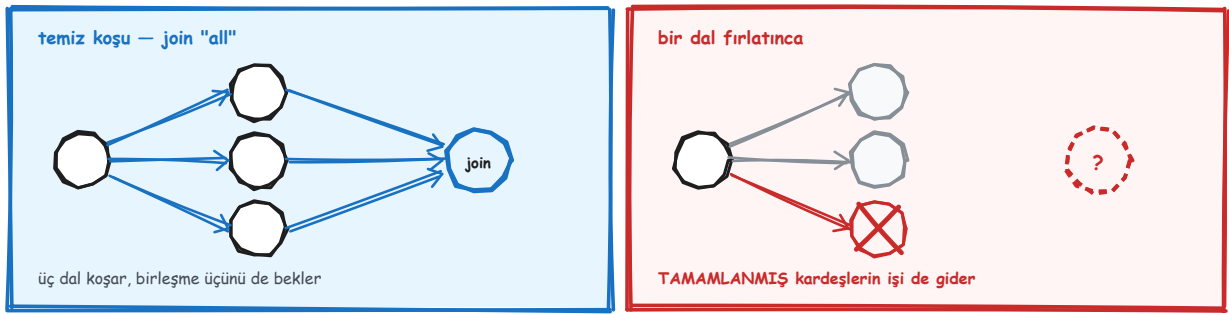
08:5398 Akışı önceden çizersin. AutoGen'in graf cevabı — ve bu projenin en önemli ölçümünün yeri.

```
builder = DiGraphBuilder()
builder.add_node(writer).add_node(reviewee)
builder.add_edge(writer, reviewee)
builder.set_entry_point(writer) # kaynak düğüm yoksa şart
flow = GraphFlow(participants=[writer, reviewee], graph=builder.build())
```

Sayfanın kapsadıkları: sıralı akış, **paralel dal + birleşme**, koşullu kenarlar (`condition=`), döngüler, `activation_group` / `activation_condition` ile karmaşık bağımlılıklar, ve `MessageFilterAgent` ile bir düğüme giden mesajı süzmek.

```
builder.add_edge(analyst_a, writer, activation_group="analysis", activation_condition="all")
builder.add_edge(analyst_b, writer, activation_group="analysis", activation_condition="all")
```

"all" → ikisini de bekler. "any" → ilk geleni alır.



ÖLÇÜLDÜ · pipeline/compare_fanin.py · aynı arıza enjeksiyonu

GraphFlow (AgentChat)

temiz 3 · sarmalayıcı arkasında 2 ·

ham hata 0-1 dal, süre sınırı dolar

core pub/sub + ClosureAgent

temiz 3 · sarmalayıcı arkasında 2 ·

ham hata 2 dal, ~3 ms

ÖLÇÜLDÜ Kaç dalın kaybedildiği **deterministik değil** — tekrarlı koşullarda 0 ve 1. Çözüm: hatayı mesaja çevirmek (`engine.ResilientClient`) ve **beklenen dal sayısını saymak**, bariyere güvenmemek.

Ve bundan güçlü bir bulgu: resmî desenler bu konuda **birbiriyle çelişiyor**. *Concurrent Agents* kuyrukla topluyor; *Mixture of Agents* `asyncio.gather(...)` ile — yani kaybın kaynağı olan yapıyla. Kılavuzun kendisi iki farklı arıza davranışı öneriyor.

12 • Bellek ile bağlam farkı

08:6220 İki ayrı iş yapar ve karıştırılırsa ajan ya unutur ya şişer.

model_context

O KONUŞMANIN son N mesajı

BufferedChatCompletionContext

verilmezse → ajan DURUMSUZ, hata da vermez

save_state bunu kaydeder

bağlam yoksa kaydedilecek sohbet de yok

Memory protokolü

KONUŞMALAR ARASI kalıcı bilgi

ListMemory • ChromaDBVectorMemory • Mem0Memory

her turda sorgulanır, dönen içerik bağlama eklenir

MemoryQueryEvent akışta görünür

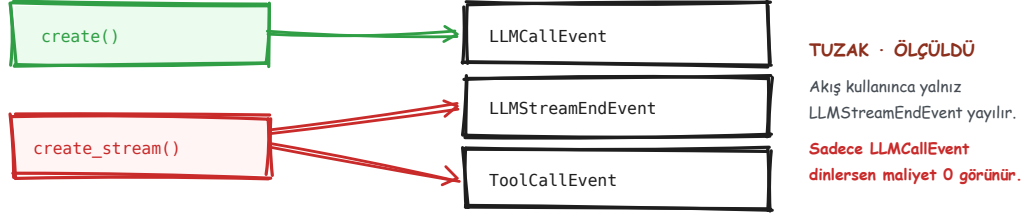
"ajan neyi hatırladı" izlenebilir

Biri konuşmanın penceresi, diğeri konuşmaların üstündeki hafıza.

13 · Günlükleme: iki logger

08:6780

```
logging.getLogger(EVENT_LOGGER_NAME) # yapılandırılmış olaylar  
logging.getLogger	TRACE_LOGGER_NAME) # insan okunur iz
```



Sıfır görünen bir maliyet, hiç görünmeyen bir maliyetten daha tehlikelidir — çünkü ona güvenirsin. → 06 §4 · pipeline/observability.py

14 · Bileşen serileştirme

08:6812

```
config = agent.dump_component()      # → JSON
agent = AssistantAgent.load_component(config)
```

Ajanı, takımı, sonlandırma koşulunu, model istemcisini JSON'a çevirip geri yükleyebilirsin. AutoGen Studio'nun kullandığı mekanizma da bu.

AutoGen'de "**skill**" diye bir soyutlama yok; en yakın karşılığı bu. Bizde kullanılmıyor — ajan topolojisi kodda tanımlı, konfigürasyondan gelen tek şey tez.

İzleme ve gözlemlenebilirlik · 08:6953

```
runtime = SingleThreadedAgentRuntime(tracer_provider=tracer_provider)
team = SelectorGroupChat([...], runtime=runtime)
```

OpenTelemetry. Bizde **henüz yok** ama yolu açık: runtime'ı zaten kendimiz kuruyoruz, tek satır. Kalan işlerin en kolayı.

15 • Özel ajanlar

08:7226 Hazır ajanlar yetmediğinde `BaseChatAgent` 'tan türetirsin. Uygulaman gereken üç şey:

```
class MyAgent(BaseChatAgent):  
    @property  
    def produced_message_types(self) -> Sequence[type[BaseChatMessage]]: ...  
    async def on_messages(self, messages, cancellation_token) -> Response: ...  
    async def on_reset(self, cancellation_token) -> None: ...
```

Sayfanın örnekleri: aritmetik yapan basit bir ajan (`ArithmeticAgent`), Gemini SDK'sını doğrudan saran bir ajan (`GeminiAssistantAgent`), ve o ajana `dump_component` / `load_component` desteği eklemek. Akışlı özel ajan için `on_messages_stream` yazılır.

16 · Üç uçtan uca örnek

08:7813 Hepsi aynı iskeleti kullanıyor: birkaç `AssistantAgent`, `RoundRobinGroupChat`, bir `TextMentionTermination`.

ÖRNEK	NE YAPIYOR	BİZE GÖRE
Seyahat planlama 08:7877	Dört rol sırayla konuşuyor, sonuncusu birleştirip "TERMINATE" diyor. Tool yok — tamamen model işbirliği	En sade örnek
Şirket araştırması 08:8045	Arama, borsa verisi, analiz. <code>FunctionTool</code> ile sarılmış fonksiyonlar, <code>max_turns</code> ile sınırlı takım	VC hattımıza en yakın — ama bizde toplayıcılar LLM'siz ve ajan katmanının <i>altında</i> ; burada tool'ları ajanlar çağırıyor
Literatür taraması 08:8357	arXiv ve Google Scholar tool'ları, sonra bir yazar ajanı raporu kuruyor	Bkz. aşağıdaki kutu

ÖLÇÜLDÜ Literatür taraması örneği, bizim `arxiv.py` toplayıcımızın **neden keşif listesinde olmadığını** iyi gösteriyor: **makale bir şirket adı taşımaz**. Literatür taraması için mükemmel, girişim keşfi için işe yaramaz — ilk canlı koşuda **30 sinyalin 30'u** hiçbir varlığa bağlanamadan düştü.

core: bir kat ařađısı

AgentChat tavan deđil. Kılavuzun bittiđi yerde aktör modeli başlıyor: kimlik, topic, abonelik, üç mesajlaşma biçimi, müdahale kapısı ve dokuz desen. Bu projede AgentChat'in çözemediđi bir sorunu buraya inerek çözdük.

17 · Neden aşağı inersin

05:73 · 455 · 543 Core'da hazır bir şey yoktur: aktörleri, mesaj tiplerini, abonelikleri kendin bağlarsın. Karşılığında **eşzamanlılık modeli** alırsın.

Kılavuzun çok-ajanlılık gerekçesi zekâ değil, ayrıştırma. Üç analistimiz üç ayrı ajan çünkü üç ayrı kaynağa bakıyorlar ve **paralel koşabiliyorlar** — daha akıllı oldukları için değil. Sayılan iki fayda: modülerlik (her ajan tek işi yapar, ayrı test edilir) ve uzmanlaşma (farklı rol, farklı model, farklı tool).

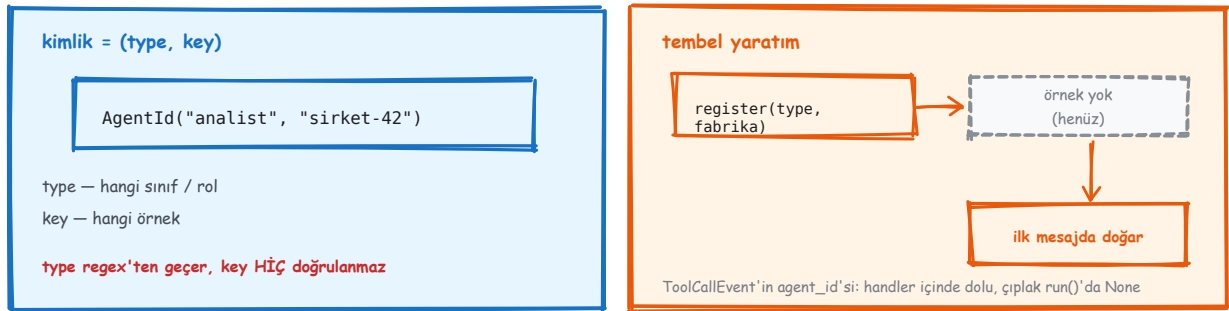
Uygulama yığını 05:543 — en altta mesajlaşma ve olay güdümlü ajanlar, üstünde geliştiricinin tanımladığı **mesaj tipleri** ve **ajan davranışları**, en üstte uygulama. Ajanlar birbirini **tanımaz**, yalnız mesaj tiplerini bilir — gevşek bağlılık buradan geliyor.

18 · Ajan kimliği ve yaşam döngüsü

05:601 Bir ajanın kimliği `(type, key)` çiftidir. Runtime ajanı **ilk mesaj geldiğinde** yaratır.

```
await MyAgent.register(runtime, "my_agent", lambda: MyAgent("demo"))
```

`register` ile **tipi ve bir fabrikayı** kaydedersin; örnek kendiliğinden oluşur. Kaç örnek doğacağını sen belirlemezsin — **gelen anahtar** belirler.



Kimlik iki parçalı, örnek tembel. `type` bir regex'ten geçiyor (`^[\\w\\-\\.]+\\Z`), `key` ise **hiç doğrulanmıyor** — kurulu pakette doğrulandı. Anahtarı dosya adına çeviren her kod bu kontrolü kendi yapmak zorunda.

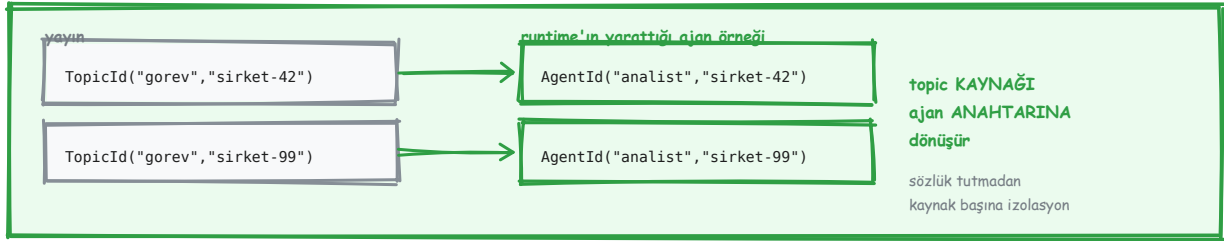
ÖLÇÜLDÜ Pratik sonucu: `ToolCallEvent` 'in taşıdığı `agent_id`, ajan runtime yönetimindeki bir handler içindeyken dolu; çıplak `agent.run()` 'da `None`. → 06 §6

19 · Topic ve abonelik

05:670 Kılavuzun **en atlanan ve en değerli sayfası**. Bir topic iki string: **tip** (mesaj kategorisi) ve **kaynak** (o kategorideki kimlik).

`TypeSubscription` bir topic **tipini** bir ajan **tipine** bağlar. Kaynak abonelikte hiç geçmez — peki nereye gider?

Topic kaynağı, ajan anahtarına dönüşür. Yani ("gorev", "sirket-42") topic'ine yayın yaparsan runtime ("analist", "sirket-42") ajanını yaratır. **Kaynak başına izole ajan örneği bedava** — sözlük tutmadan, temizlik kodu yazmadan. Çok kiracılı yapının hazır mekanizması bu.



CANLI KOŞULDU · autogen-core 0.7.5 · üç yayın, iki örnek

```
publish TopicId('gorev','sirket-42') -> key='sirket-42'
publish TopicId('gorev','sirket-99') -> key='sirket-99'
publish TopicId('gorev','sirket-42') -> key='sirket-42' (aynı örnek)
```

kayıtlı örnek: 2

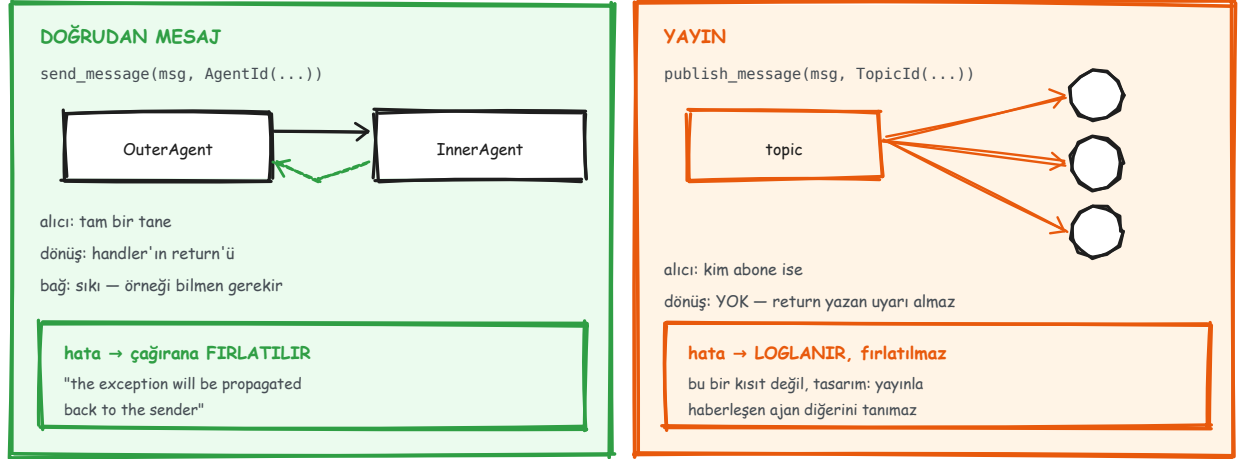
Üç yayın, iki örnek — ikinci `sirket-42` yeni ajan doğurmadı, aynısını yeniden kullandı. Koşturularak doğrulandı.

SENARYO	NASIL	BİZDE
Tek kiracı, tek topic	Hepsi aynı topic tipi, kaynak sabit "default"	fanin.py — tek kullanıcıli araç
Tek kiracı, çok topic	Ajan başına ayrı topic tipi, kaynak aynı	—
Çok kiracı	Kaynak veriye bağlı — oturum başına izolasyon	gateway/sessions.py — kaynak = oturum kimliği

20 · Üç mesajlaşma biçimi

05:1108 Aradaki fark adresleme değil — **hata davranışı**. Bu tek satır, bir tasarım kararıdır.

Biçim	NASIL	NE ZAMAN
Doğrudan mesaj	<code>send_message(msg, AgentId(...))</code>	Cevap bekliyorsan (RPC gibi)
Yayın	<code>publish_message(msg, TopicId(...))</code>	Kim dinliyorsa duysun
Cevap	handler'dan <code>return</code>	Doğrudan mesajın karşılığı



Yayının **cevabı yoktur** — kısıt değil, tasarım: yayınla haberleşen ajanlar birbirini tanımak zorunda kalmaz. Sayfa ayrıca `CancellationToken` ile çalışan bir işi iptal etmeyi anlatıyor.

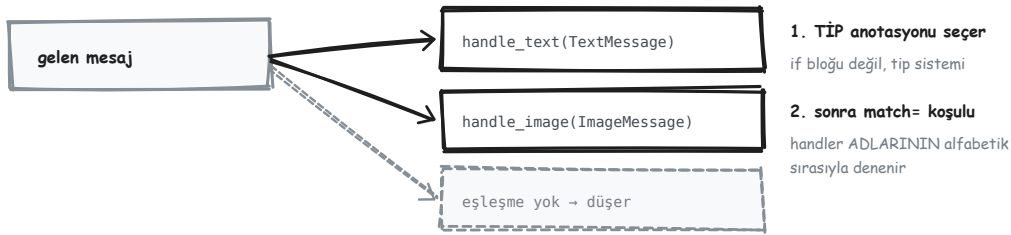
ÖLÇÜLDÜ Bu projede `send_message` hiç kullanılmıyor — kod tabanında sıfır geçiş, her şey yayın. Bu bir eksik değil, tutarlılık: bizim disipliniimiz *"a failure is published, not raised."* Doğrudan mesajlaşma o kuralı tersine çevirirdi.

21 · RoutedAgent: tip önce, sonra koşul

05:894

```
class MyAgent(RoutedAgent):  
    @message_handler  
    async def handle_text(self, message: TextMessage, ctx: MessageContext) -> None: ...  
  
    @message_handler  
    async def handle_image(self, message: ImageMessage, ctx: MessageContext) -> None: ...
```

Bir ajanın birden çok handler'ı olabilir; hangisinin çalışacağını **tip anotasyonu** belirler — **if** bloğu değil, tip sistemi. Union tipi de kullanılabilir. İkinci eleme **match=** ile yapılır ve koşullar **handler adlarının alfabetik sırasıyla** denir.



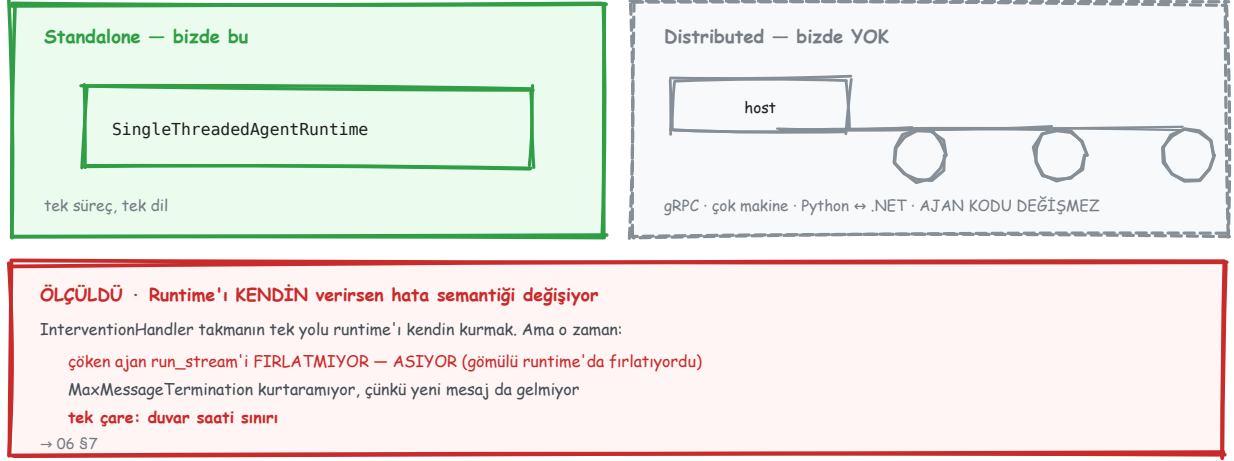
TUZAK — bizi yakaladı · kayıt sırasında çıplak NameError

Tip çıkarımı `get_type_hints()` ile. ``from __future__ import annotations`` varken `MessageContext`'i fonksiyon içinde import edersen KAYITTA patlar. Ve parametre adları bağlayıcı: ``message`` ve ``ctx`` olmak zorunda.

Yönlendirme iki aşamalı. Sıraya güveniyorsan handler adlarını ona göre seç — bu, kodun okunuşundan çıkarılamayan bir davranış.

22 · Runtime: iki tip, ve kendininkini kurmanın bedeli

05:490 · 1742 Kritik vaat: **ikisi aynı API'yi sunuyor**, ajan kodunu değiştirmeden geçebiliyorsun.



Dağıtık tarafta `GrpcWorkerAgentRuntimeHost` (koordinatör) + `GrpcWorkerAgentRuntime` (işçiler); diller arası çalışma (Python ↔ .NET) da bu sayfada. Bizde yok — tek süreç yetiyor ve hacim sorusu cevaplanmadan gereksiz karmaşa.

Bu, belgedeki en sinsî ölçüm. Müdahale kapısı takmak istiyorsan runtime'ı kendin kurmak zorundasın — ve kurduğun anda çöken bir ajanın davranışı sessizce değişiyor. Hata artık yüzeye çıkmıyor, iş **asılı kalıyor**. Sonlandırma koşulu da kurtaramıyor, çünkü onu tetikleyecek mesaj hiç gelmiyor.

23 · Bileşenler: bağlam, tool, workbench

Model bağlamı · 05:2341

SINIF	NE YAPAR
UnboundedChatCompletionContext	Her şeyi tutar
BufferedChatCompletionContext(buffer_size=N)	Son N mesajı
HeadAndTailChatCompletionContext	Baş + son
TokenLimitedChatCompletionContext	Token sınırı

İnce nokta: `buffer_size` **mesaj** sayar, token değil. Uzun bir sohbette maliyeti sınırlayan şey bu *değildir* — onun için `TokenLimited...` var.

Tool'lar · 05:2473

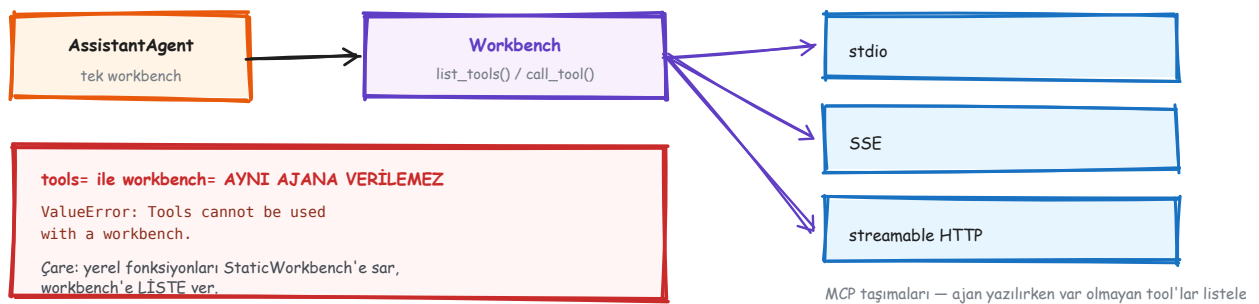
`FunctionTool` şemayı **imzadan ve docstring'den** üretir — yani docstring dokümantasyon değil, **arayüz**.

```
async def get_stock_price(ticker: str, date: Annotated[str, "YYYY/MM/DD"]) -> float:
    """Get the stock price."""
    return random.uniform(10, 200)
```

ÖLÇÜLDÜ **Ve bu, bizi ısıran yerd:** şema imzadan üretildiği için **gövde hiç koşmadan** geçerli görünür. On dört tool'un şeması doğruydu, üçünün gövdesi olmayan bir API çağırıyordu ve şema testleri **geçiyordu**. *İmzadan üretilen bir şeyi test etmek, o şeyin çalıştığını test etmek değildir.*

Workbench ve MCP · 05:2841

Workbench bir **tool kaynağı** — tek tek tool yerine "bana hangi tool'ların olduğunu söyle" diyebildiğin nesne. Uzak bir MCP sunucusu tam olarak bu: ajan yazılırken var olmayan tool'ları listeleyebilir.



Üç taşıma, tek arayüz. Bu projede OpenClaw köprüsü ve DeepWiki, ikisi de `McpWorkbench` olarak aynı listeye giriyor.

Kod yürütücüler · 05:3054

```
LocalCommandLineCodeExecutor(work_dir=...) # yerel — dikkat
DockerCommandLineCodeExecutor(image=...) # izole
```

Kılavuz yerel yürütücü için **açık uyarı** veriyor: modelin ürettiği kodu izolesiz çalıştırmak risklidir; Docker önerilir. Bizde yok — hatta kod yürütülmüyor.

24 • Müdahale kapısı

05:6534 Runtime'a takılan tek kapı. Her mesaj buradan geçer; `DropMessage` döndürerek düşürebilirsin.



Bu projede `AuditingInterventionHandler` — denetim kaydı burada yazılıyor. Kapının bedeli §22'de: takmak için runtime'ı kendin kurmalısın.

Günlükleme ve olaylar • 05:1534

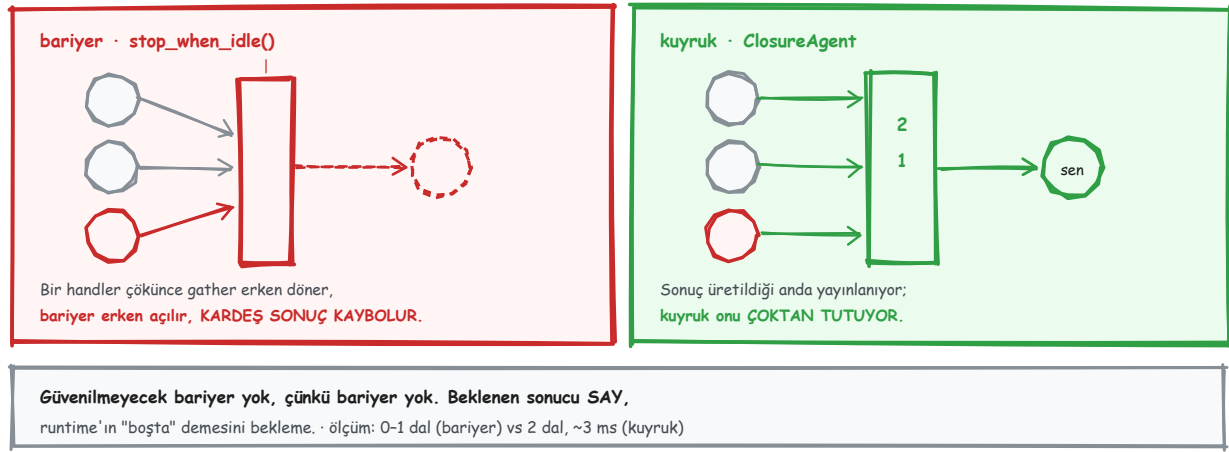
Olay akışında `LLMCallEvent`, `LLMStreamEndEvent`, `ToolCallEvent`, `MessageEvent`, `MessageDroppedEvent`, `MessageHandlerExceptionEvent` var.

ÖLÇÜLDÜ İkinci tuzak, ve sessiz olanı: `LLMCallEvent` alanlarını öznitelik yapıyor ama `ToolCallEvent` her şeyi `.kwargs` **sözlüğünde** tutuyor. `event.tool_name` yazarsan `AttributeError` alırsın — ve o hata `logging.Handler` içinde **yutulur**: olay hiç kaydedilmez, hata da görünmez. → 06 §6

25 · Dokuz desen – ve toplama meselesi

05:3209 Ana ayrım: **önceden çizili** (sequential, graph) ve **konuşmadan doğan** (group chat, handoff).

DESEN	ÇEKİRDEĞİ	BİZDE
Concurrent Agents 3236	Aynı topic'e abone ajanlar + ClosureAgent kuyruğu	✓ fanin.py
Sequential Workflow 3504	Her ajan bir öncekinin çıktısını alır	kodla, ajanla değil
Group Chat 3772	Ortak thread + konuşmacı seçici	X
Handoffs 4349	Ajan devir tool'u çağırır	X — en pahalı (334)
Mixture of Agents 4989	Katmanlı işçiler + orkestratör	X
Multi-Agent Debate 5358	Seyrek topoloji, N ajan × R tur, çoğunluk oyu	X
Reflection 5822	Üretici + eleştirmen döngüsü	X
Code Execution 6188	Group chat + kod yürütücü	X



Toplamanın iki yolu ve aralarındaki fark, bu projenin en pahalı dersi.

Ve buradan çıkan en güçlü bulgu: resmî desenler bu konuda **birbiriyle çelişiyor**. *Concurrent Agents* kuyrukla topluyor; *Mixture of Agents* `asyncio.gather(...)` ile — yani sessiz kardeş kaybının kaynağı olan yapıyla. Kılavuzun kendisi iki farklı arıza davranışı öneriyor, ve hangisini seçtiğin ancak bir dal çöktüğünde belli oluyor.

Reflection'ın söylenmeyen riski

Kılavuz üretici-eleştirmen döngüsü için **durma ölçütü önermiyor**. Döngüyü olduğu gibi alırsan faturayı sınırlandıran hiçbir şey kalmıyor — deseni kopyalamanın en riskli yeri burası.

26 · Kullanmadığımız üç yüzey

Bilerek dışarıda bırakıldı; hangisinin ne zaman gerekeceği yazılı.

YÜZEY	NE VERİYOR	NEDEN YOK
Dağıtık runtime 05:1742	Ajanlar makinelere dağılır, ajan kodu değişmez	Tek süreç yetiyor. Core'a taşınmanın asıl kazancı bu olurdu — bu bölümden sonra tek satırlık değişiklik
Bileşen yapılandırma 05:1888	<code>dump_component()</code> / <code>load_component()</code> — JSON'a çevir, geri yükle	Topoloji kodda tanımlı; konfigürasyondan gelen tek şey tez
OpenTelemetry 05:1647	<code>tracer_provider</code> ver, mesajlar ve model çağrılarını span olur	Runtime'ı zaten kendimiz kuruyoruz — tek satır . Kalan işlerin en kolayı

AutoGen'de **"skill" diye bir soyutlama yok**; en yakın karşılığı bileşen serileştirme, ve AutoGen Studio'nun kullandığı mekanizma da bu.

27 · Cookbook: on dört tarif

05:6461 Baştan sona okunacak bir bölüm değil — **derdine göre girilir**. Kılavuzun en az bilinen ve pratikte en çok işe yarayan kısmı burası.

DERDİN	TARİF	satır
Ajanım gerçek dünyada bir şey yapacak	→ Tool yürütme için kullanıcı onayı	05:6638
Faturayı görmem lazım	→ LLM kullanımını logger ile takip	05:8840
Veri dışarı çıkamaz	→ Yerel LLM: LiteLLM ve Ollama	05:8030
Anahtar config'de duramaz	→ Azure OpenAI + AAD kimlik doğrulama	05:6490
Zaten LangGraph/LlamaIndex var	→ ... destekli ajan — ikisi birlikte	05:7482
Çok kiracılı olacak	→ Topic abonelik senaryoları	05:8233
Cevap şemaya uymalı	→ GPT-4o ile yapısal çıktı	05:8745
Nerede yavaşlıyor bilmiyorum	→ Kodunu yerelde izleme (OTel + Jaeger)	05:8192

Sekiz yaygın dert ve karşılığı. Kalan altı tarif aşağıdaki tabloda.

Her takımın er ya da geç ihtiyaç duyacağı dört tanesi

1 · Tool yürütme için kullanıcı onayı 05:6638

Müdahale kapısının asıl kullanımı: `DropMessage` döndürerek bir tool çağrısını **engellemek**.

Buradaki cümle, bu tarifi diğerlerinden ayıran şey: **onay kapısı ajanın uyum göstermeyi seçmesine değil, runtime'a dayanır**. Sistem istemine "önce sor" yazmak bir ricadır; `InterventionHandler` bir kapıdır. Ajanın kandırılması bu kapıyı açmaz.

Bizde: bağlı ve testli. Tool'ların çoğu salt-okunur olduğu için gözlemci modda duruyordu; ilk mutasyon yapan tool (OpenClaw'a mesaj göndermek) gelince kapı devreye girdi. `gateway/approval.py`

2 · LLM kullanımını logger ile takip 05:8840

```
class LLMUsageTracker(logging.Handler):
    def emit(self, record):
        if isinstance(record.msg, LLMCallEvent):
            self._prompt_tokens += record.msg.prompt_tokens
            self._completion_tokens += record.msg.completion_tokens

logging.getLogger(EVENT_LOGGER_NAME).handlers = [LLMUsageTracker()]
```

ÖLÇÜLDÜ Kılavuzdaki bu tarif eksik. Yalnız `LLMCallEvent` dinliyor; `create_stream()` onu **yaymıyor**. Akış açıksa bu sayaç **0** gösterir ve sen ona güvenirsin. `LLMStreamEndEvent` de dinlenmeli — bizim `observability.py` 'miz tarifi bu genişletilmiş hâli.

3 · Yerel LLM'ler: LiteLLM ve Ollama 05:8030

`OpenAIChatCompletionClient` + `base_url` ile LiteLLM proxy'sine ya da Ollama'ya bağlanma. Verinin kurumdan çıkamadığı her takımın yolu bu — **ve `model_info` vermek zorunda**

kalacağın yer de tam burası (§4'teki tuzak).

4 • LangGraph / LlamaIndex destekli ajan 05:7482 • 7662

Bir LangGraph grafinı ya da bir LlamaIndex RAG hattını `RoutedAgent` içine sarmak. **Framework seçmek zorunda değilsin.**



Dışarıyı aktör modeli, içerisi ne istersen. "AutoGen mı LangGraph mı" sorusunun cookbook'taki cevabı: *ikisi de*.

Kalan tarifler

TARİF	NE İÇİN	SATIR
Azure OpenAI + AAD kimlik doğrulama	API anahtarı yerine AAD token'ı — <code>get_bearer_token_provider(DefaultAzureCredential())</code> : Anahtarın config dosyasında duramadığı kurumlar için	05:6490
Müdahale ile sonlandırma	Mesaj hattına teslimden önce oturmak. Uyarı: handler <code>None</code> döndürürse runtime uyarı basar — mesajı ya da <code>DropMessage</code> 'i açıkça döndürmelisin	05:6534
Ajanla sonuç çıkarma	<code>ClosureAgent</code> ile yayınlanan sonuçları kuyruğa toplama — §25'teki toplama mekanizmasının tek başına anlatımı	05:6825
OpenAI Assistant ajanı	Assistants API'sini bir ajana sarma: thread yönetimi, dosya arama, kod yorumlayıcı	05:6914
Topic abonelik senaryoları	§19'un çalışan örneklerle hâli. Çok kiracılı yapı kuracaksan okunacak sayfa bu	05:8233
GPT-4o ile yapısal çıktı	<code>response_format</code> ile Pydantic şeması dayatma. AgentChat'in <code>output_content_type</code> 'i bunun üstüne kuruluyor	05:8745
Kodunu yerelde izleme	OpenTelemetry'yi yerelde kurup Jaeger gibi bir toplayıcıya bağlama	05:8192

28 · Kılavuzun kendi kapattığı sorular

05:8914 SSS bölümünün kapattığı konular, ve bu projede her birinin ne çıktığı.

SORU	BU PROJEDEKİ CEVABI
Bir ajanı nasıl uzaktan çağırırım?	Dağıtık runtime (gRPC). Kurulu değil — ama §22'deki vaat gereği ajan kodu değişmeden geçilir. Core'a taşınmanın asıl kazancı bu
Runtime'lar arasındaki fark?	API aynı, hata semantiği değil . Kendi runtime'ını kurunca çöken ajan fırlatmıyor, asıyor — ölçüldü, §22
Aynı ajanın birden çok örneği olur mu?	Olur, ve bedava: topic kaynağı ajan anahtarına dönüşüyor. §19
Mesaj kaybolur mu?	Evet . Yayın handler'ı çökerse loglanır ama fırlatılmaz; bariyere güvenirsen tamamlanmış kardeş sonuçlar da gider. §20, §25
Hata yönetimi nasıl olmalı?	Bizim kuralımız: <i>"a failure is published, not raised"</i> — hatayı mesaja çevir, beklenen sonuç sayısını say
AgentChat mi core mu?	AgentChat'le başla; ısırdığı yerde in. Bizde ısırın yer paralel dal kaybıydı (§11) ve çözümü core'daydı (§25)

Kılavuzun kendi tavsiyesi de bu yönde ve belgenin tamamının özeti sayılır: core'a inmek bir başarısızlık değil, *tasarlanmış bir kaçış yolu*. AgentChat'in var olma sebebi core'u her seferinde yazmak zorunda kalmamandır — ama tavan olduğunu hiç iddia etmiyor.

29 · Ölçümler, tek sayfada

Bu belgedeki **ÖLÇÜLDÜ** etiketli her şey, ölçüm dosyasıyla.

BULGU	SAYI	DOSYA
Desen seçiminin token farkı	% 63,7 — Selector 204 · GraphFlow 270 · RoundRobin 274 · Swarm 334	poc/kiyas.py
GraphFlow ham hata altında kurtarılan dal	0-1 (deterministik değil), süre sınırı dolar	pipeline/compare_fanin.py
core pub/sub + ClosureAgent, aynı arıza	2 dal, ~ 3 ms	pipeline/compare_fanin.py
Akıшта maliyet sayacı	create_stream() yalnız LLMStreamEndEvent yayar	pipeline/observability.py
model_info zorunluluğu	OpenAI-uyumlu endpoint'te ilk çağrıda ValueError	pipeline/probe_llm.py
arXiv sinyallerinin bağlanma oranı	30 / 30 düştü — makale şirket adı taşımıyor	ilk canlı koşu
mcp bağımlılığı üst sınırsız	MCP SDK 2.0 → ImportError: RequestContext	requirements.txt

30 · Bu projede ne kullanılıyor

Kod tabanı `grep` 'lendi, ezberden değil. İki katmanın da yüzeyinden kullandığımız ve kullanmadığımız her şey.

core

YÜZEY	DURUM	NEREDE
<code>AgentId(type, key)</code>	✓ merkezi	<code>sessions.py</code> — <code>self.id.key</code> oturum kimliğinin <i>kendisi</i>
Topic ve abonelik	✓ üç senaryodan iki'si	<code>fanin.py</code> sabit kaynak · <code>sessions.py</code> oturum kaynağı
<code>RoutedAgent</code> + <code>@message_handler</code>	✓	<code>fanin.py</code> , <code>channels.py</code> , <code>gateway/sessions.py</code>
<code>ClosureAgent</code>	✓ — toplama kuyruğu	<code>fanin.py</code>
<code>SingleThreadedAgentRuntime</code>	✓ iki tane	<code>fanin.py</code> kısa ömürlü · <code>gateway/runtime.py</code> uzun ömürlü
<code>publish_message</code>	✓	<code>fanin.py</code> , <code>gateway/sessions.py</code>
<code>send_message</code> (doğrudan)	✗ sıfır geçiş — bilinçli	yayın "fırlatmaz, loglar" disiplini korunuyor
<code>@type_subscription</code> dekoratörü	✗ — dördü de elle	<code>add_subscription(...)</code> · kayıt ve abonelik ayrı dursun diye
<code>InterventionHandler</code> · <code>DropMessage</code>	✓	<code>observability.py</code>
<code>EVENT_LOGGER_NAME</code> + üç olay	✓	<code>LLMCallEvent</code> , <code>LLMStreamEndEvent</code> , <code>ToolCallEvent</code>
<code>FunctionTool</code> · <code>StaticWorkbench</code> · <code>McpWorkbench</code>	✓ iki taşıma	stdio (OpenClaw) + streamable HTTP (DeepWiki)
Model bağlamı	✓ genişletilmiş	<code>BufferedChatCompletionContext</code> + kendi <code>CompactingChatCompletionContext</code> 'imiz
Dokuz desen	✓ yalnız Concurrent Agents	<code>fanin.py</code> · kalanı <code>poc/</code> 'ta ölçüm için
Dağıtık runtime (gRPC)	✗	<code>grpcio</code> kurulu değil
Kod yürütücüler	✗	hatta kod yürütülmüyor

AgentChat

YÜZEY	DURUM	NEREDE
AssistantAgent	✓	pipeline/agents/
GraphFlow + DiGraphBuilder	✓ join activation_condition="all"	pipeline/graph.py
MaxMessageTermination	✓ — 11 koşuldan 1 tanesi	her takımda sigorta
StructuredMessage / output_content_type	✓	agents/triage.py , agents/analysts.py , engine.py
save_state / load_state	✓	conversation.py , gateway/context_engine.py
TaskResult	✓	fanin.py
RoundRobin · Selector · Swarm · MagneticOne	✗ — beş takımdan 1'i üründe	yalnız poc/kiyas.py 'de ölçüm için
Handoff / devirler	✗	—
Memory protokolü	✗ — kendi Markdown belleğimiz var	gateway/memory.py
UserProxyAgent	✗ — insan döngüsü HTTP üstünden	server.py
dump_component / load_component	✗	topoloji kodda tanımlı
OpenTelemetry	✗ — yolu açık, tek satır	—
CodeExecutorAgent	✗	hatta kod yürütme yok

Özet: AgentChat'in beş takım tipinden **bir** tanesini, on bir sonlandırma koşulundan **bir** tanesini kullanıyoruz. Bu bir eksik değil, bir **tez**: *AutoGen'i ürün olarak değil, desen kütüphanesi ve ölçüm zemini olarak kullanıyoruz*. Ölçüm bunu doğruluyor — kullanmadıklarımızın çoğu poc/ 'ta koştı, sayısı alındı, üründe bırakılmadı.

Okurken aklında tutulacak dört şey

1. **Kılavuz "nasıl yapılır"ı anlatıyor, "ne zaman bozulur"u anlatmıyor.** Bu projede bulunan 13 incelik hiçbir sayfada yazmıyor. → 06
2. **Kod örnekleri gpt-4o varsayıyor.** Başka bir endpoint kullanıyorsan model_info vermek ve hangi yeteneklerin gerçekten desteklendiğini **ölçmen** gerekir. → pipeline/probe_llm.py
3. **AgentChat tavan değil.** Kılavuzun bittiği yerde autogen_core başlıyor. Bu projede AgentChat'in çözemediği bir sorunu (paralel dal kaybı) bir kat aşağı inerek çözdük. → §11
4. **AutoGen bakım modunda.** Bu kılavuz doğru ama donmuş; halefi Microsoft Agent Framework ve AutoGen'den bir göç kılavuzu var.

Kaynak: docs/08-autogen-agentchat-user-guide.md (microsoft/autogen, MIT) · Türkçe özet docs/10-agentchat-turkce.md · Kod ve API adları bilinçli olarak İngilizce bırakıldı. Satır atıfları 08:NNNN biçiminde, tam metne o satırdan girilir.